
Lecture 18: Ensemble Learning Part 2

Md. Shahriar Hussain
ECE Department, NSU



Boosting

- *Boosting (originally called hypothesis boosting) refers to any Ensemble method that can combine several weak learners into a strong learner.*
- The general idea of most boosting methods is to train predictors sequentially, each trying to correct its predecessor.
- There are many boosting methods available, but by far the most popular are *AdaBoost (Adaptive Boosting) and Gradient Boosting*.



Boosting – General idea

- Boosting → sequentially train predictors that each try to predict examples that were hard for previous predictors
- Original Algorithm:
 - Train classifier 1: use random subset of examples without replacement
 - Train classifier 2: use a second random subset of examples without replacement and add 50% of examples misclassified by classifier 1
 - Train classifier 3: use examples that classifiers 1 and 2 disagree on
 - Predict using majority vote from 3 classifiers



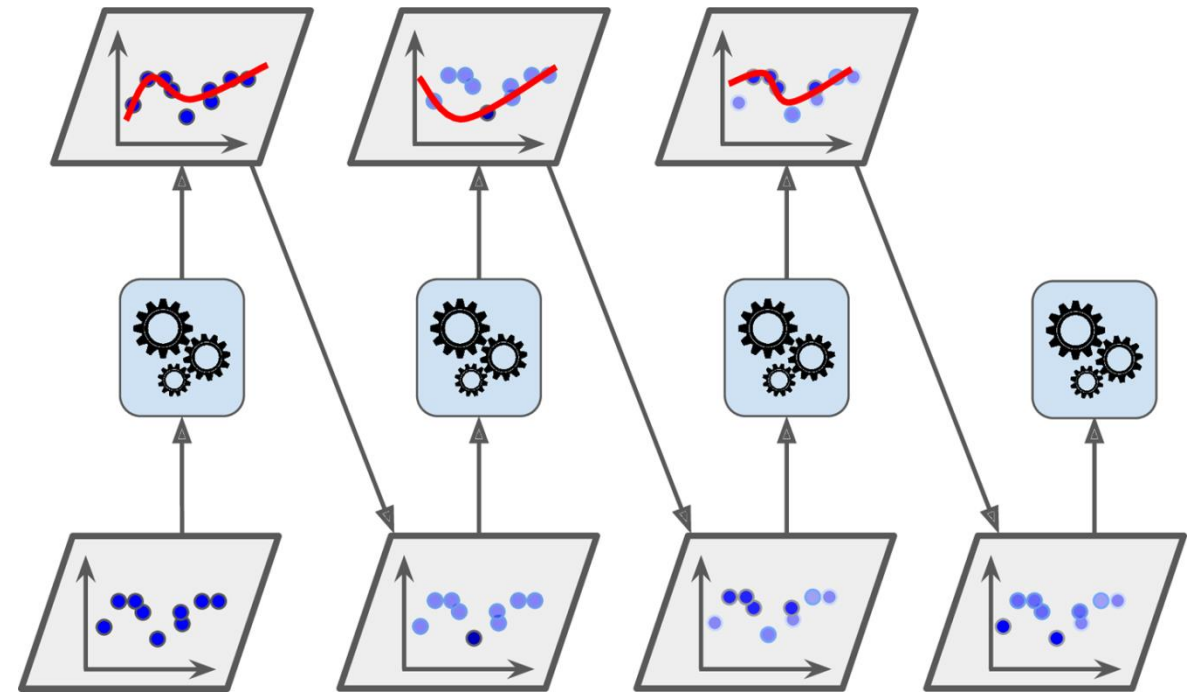
AdaBoost

- One way for a new predictor to correct its predecessor is to pay a bit **more attention** to the training instances that the predecessor **underfitted**.
- This results in new predictors focusing more and more on the hard cases. This is the technique used by AdaBoost.
- When training an AdaBoost classifier, the algorithm **first trains a base classifier** (such as a Decision Tree) and uses it to make predictions on the training set.



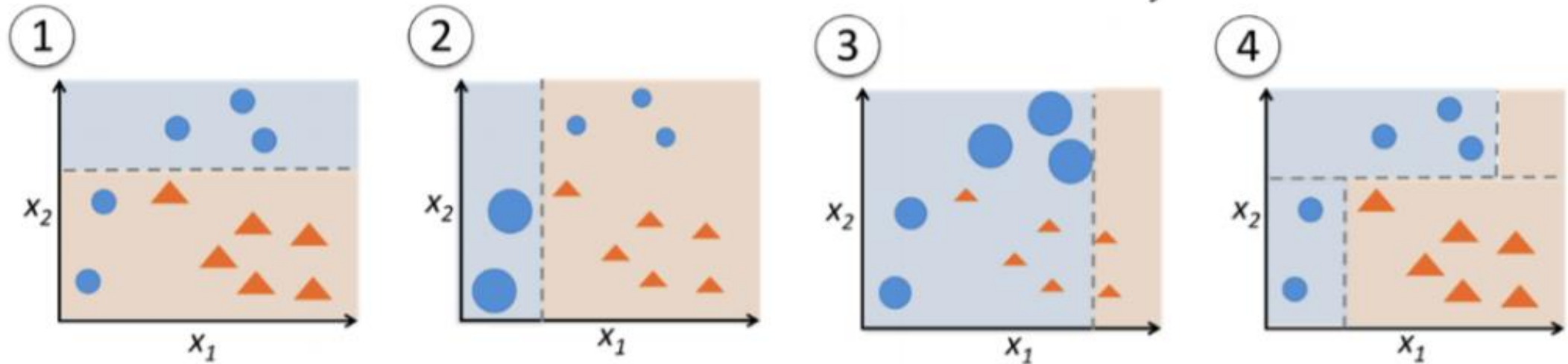
AdaBoost

- The algorithm then **increases the relative weight of misclassified training instances**.
- Then it trains a second classifier, using the updated weights, and again makes predictions on the training set, updates the instance weights, and so on



AdaBoost Steps

1. Assign equal weights to all examples
2. Assign larger weights to previous misclassifications, and smaller weights to previous correct classifications
3. Assign larger weights to training samples C1 and C2 disagree on, and smaller weights to previously correct classifications
4. Predict with weighted majority vote



Adaboost Example

Sample indices	x	y	Weights	Predicted Y ($x \leq 3.0$)	Correct?	Updated weights
1	1.0	1	0.1	1	Yes	0.088
2	2.0	1	0.1	1	Yes	0.088
3	3.0	1	0.1	1	Yes	0.088
4	4.0	-1	0.1	-1	Yes	0.088
5	5.0	-1	0.1	-1	Yes	0.088
6	6.0	-1	0.1	-1	Yes	0.088
7	7.0	-1	0.1	1	No	0.127
8	8.0	-1	0.1	1	No	0.127
9	9.0	-1	0.1	1	No	0.127
10	10.0	-1	0.1	-1	Yes	0.088



AdaBoost Algorithm

- At first, each instance weight $w^{(i)}$ is *initially* set to **1/m** equally.
- First predictor is trained, and its weighted error rate r_1 is *computed on* the training set
- *Weighted error rate of the j^{th} predictor,*

$$r_j = \frac{\sum_{i=1}^m w^{(i)} \mathbb{I}_{\hat{y}_j^{(i)} \neq y^{(i)}}}{\sum_{i=1}^m w^{(i)}}$$

where $\hat{y}_j^{(i)}$ is the j^{th} predictor's prediction for the i^{th} instance

AdaBoost Algorithm

- After that the predictor's weight α_j is computed

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

- where η is the learning rate hyperparameter (defaults to 1).
- The more accurate the predictor is, the higher its weight will be. If it is just guessing randomly, then its weight will be close to zero.
- However, if it is most often wrong (i.e., less accurate than random guessing), then its weight will be negative



AdaBoost Algorithm

- Next, the AdaBoost algorithm updates the instance weights which eventually boosts the weights of the misclassified instances

$$\text{for } i = 1, 2, \dots, m$$
$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp(\alpha_j) & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases}$$

- Then all the instance weights are normalized by dividing total weight, $\sum_{i=1}^m w^{(i)}$
- Finally, a new predictor is trained using the updated weights, and the whole process is repeated



AdaBoost Algorithm

- The algorithm stops when the desired number of predictors is reached, or when a perfect predictor is found
- To make predictions, AdaBoost simply computes the predictions of all the predictors and weighs them using the predictor weights α_j . *The predicted class is the one that receives the majority of weighted votes*



Adaboost Example- Updating Weight

- First time the base weight was, $1/m = 1/10 = 0.1$
- 1. Compute error rate (sum of misclassified examples' weight/Total weight):

$$R_j = (0.1 + 0.1 + 0.1) / 0.1 * 10 = 0.3 / 1 = 0.3$$

- 2. Compute predictors weight,

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

$$= 1 * \log((1 - 0.3) / 0.3) = 0.3679 \quad [\text{consider } \eta = 1]$$

Sample indices	Correct ?	Updated weights
1	Yes	
2	Yes	
3	Yes	
4	Yes	
5	Yes	
6	Yes	
7	No	
8	No	
9	No	
10	Yes	



Adaboost Example- Updating Weight

- Update weight vector:

for $i = 1, 2, \dots, m$

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp(\alpha_j) & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases}$$

For correct predicted one:

$$w^{(i)} = 0.1$$

For incorrect predicted one:

$$w^{(i)} = 0.1 * e(0.3679) = 0.1444$$

Sample indices	Correct ?	Updated weights
1	Yes	
2	Yes	
3	Yes	
4	Yes	
5	Yes	
6	Yes	
7	No	
8	No	
9	No	
10	Yes	



Adaboost Example- Updating Weight

- Find total weight= $0.1*7+0.1444*3=1.133$
- Updated normalized weight:
 - For correct predicted one: $0.1/1.133=0.088$
 - For incorrect predicted one: $0.1444/1.133=0.127$

Sample indices	Correct ?	Updated weights
1	Yes	0.088
2	Yes	0.088
3	Yes	0.088
4	Yes	0.088
5	Yes	0.088
6	Yes	0.088
7	No	0.127
8	No	0.127
9	No	0.127
10	Yes	0.088



Adaboost Hyperparameter

- The following code trains an AdaBoost classifier based on 200 *Decision Stumps* using Scikit-Learn's AdaBoostClassifier class

```
from sklearn.ensemble import AdaBoostClassifier
ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=200,
    algorithm="SAMME.R", learning_rate=0.5)
ada_clf.fit(X_train, y_train)
```



Gradient Boosting

- Another very popular boosting algorithm is *Gradient Boosting*.
- *Just like AdaBoost*, Gradient Boosting works by sequentially adding predictors to an ensemble, each one correcting its predecessor.
- However, **instead of tweaking the instance weights** at every iteration like AdaBoost does, this method tries to fit the **new predictor to the *residual errors* made by the previous predictor**.



Gradient Boosting

- First, let's fit a DecisionTreeRegressor to the training set

```
from sklearn.tree import DecisionTreeRegressor
tree_reg1 = DecisionTreeRegressor(max_depth=2)
tree_reg1.fit(X, y)
```

- Next, we'll train a second DecisionTreeRegressor on the residual errors made by the first predictor:

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2)
tree_reg2.fit(X, y2)
```

- Then we train a third regressor on the residual errors made by the second predictor:

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2)
tree_reg3.fit(X, y3)
```



Gradient Boosting

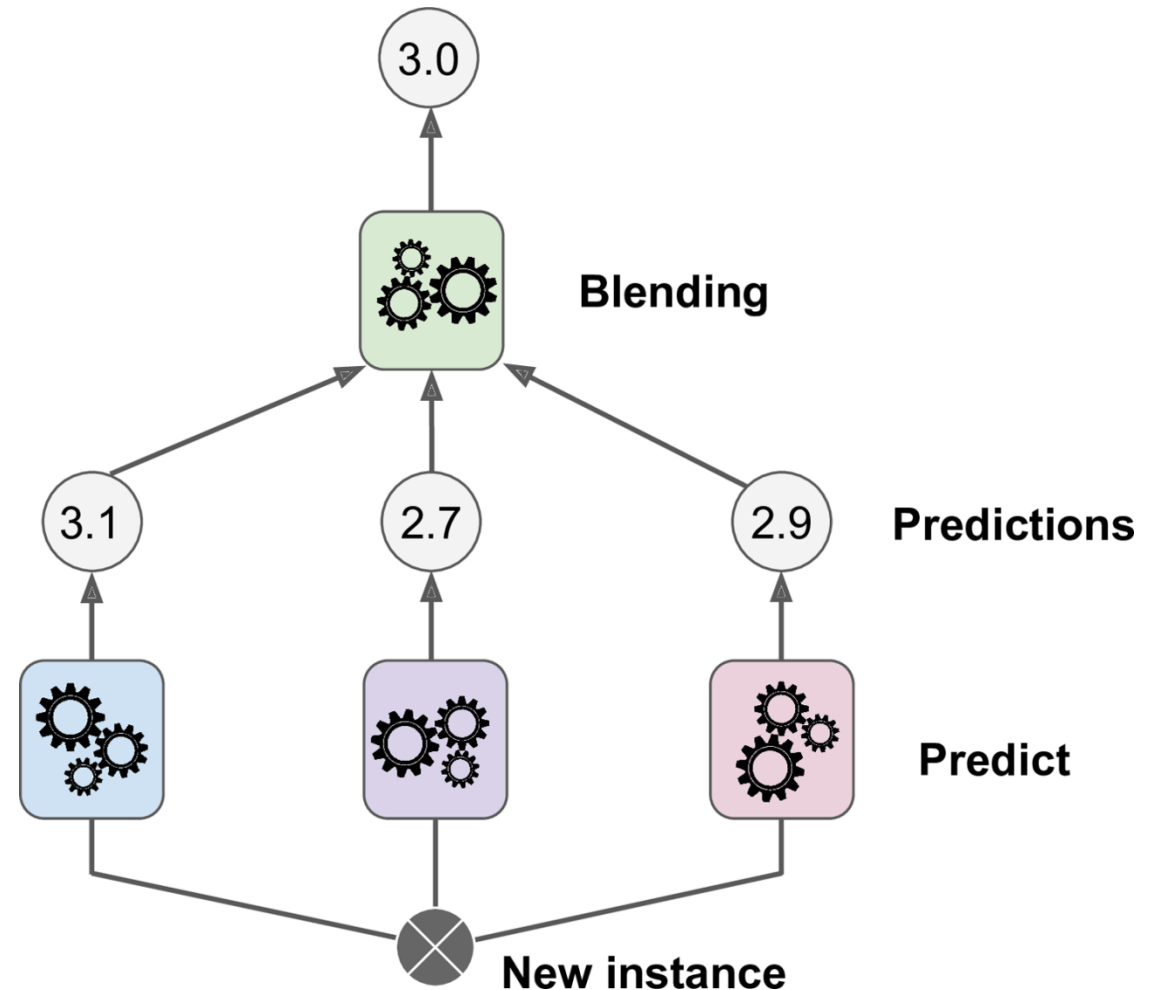
- Now we have an ensemble containing three trees. It can make predictions on a new instance simply by adding up the predictions of all the trees:

```
y_pred = sum(tree.predict(X_new) for tree in (tree_reg1,  
tree_reg2, tree_reg3))
```



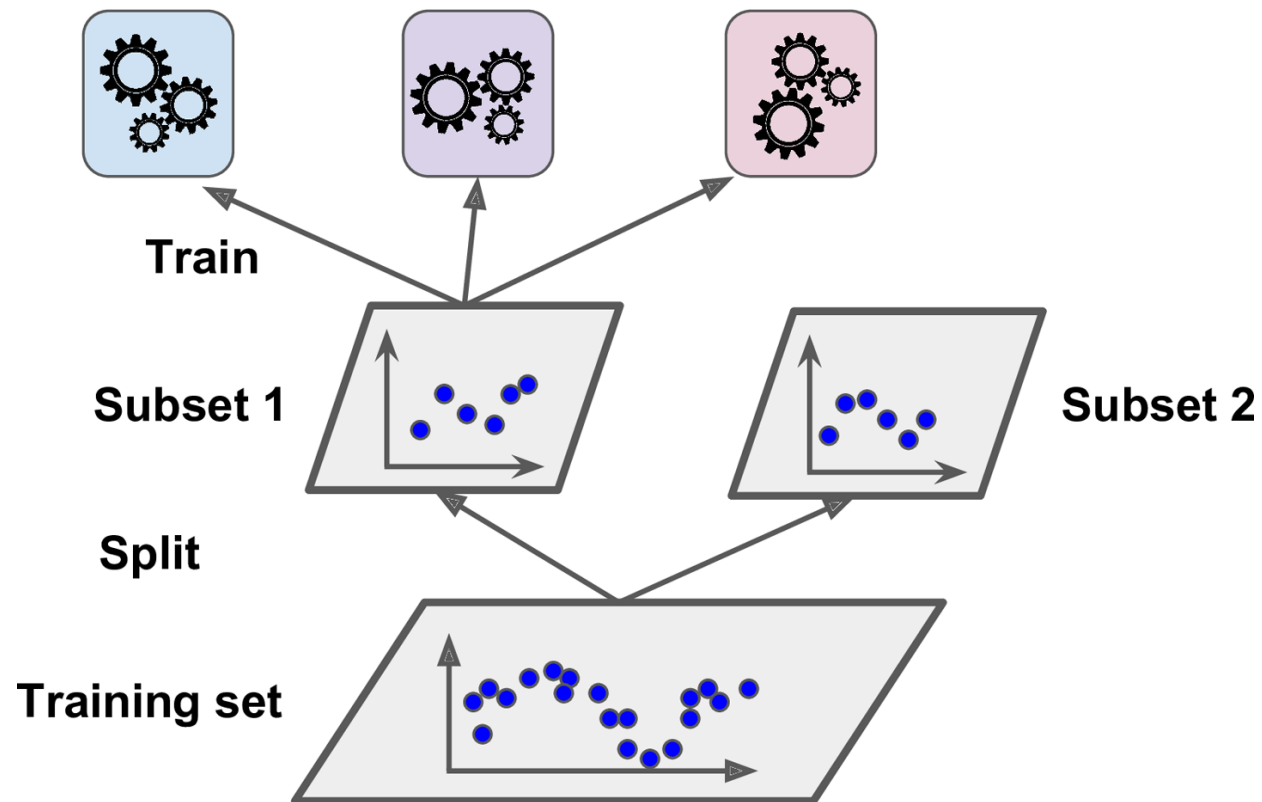
Stacking

- Stacked Generalization
 - Instead of using hard voting to aggregate the predictions of all predictors, why not train a model to perform the aggregation itself?
 - Final predictor (called blender, or meta learner) takes predictions made by each of the ensemble predictors and makes a final prediction



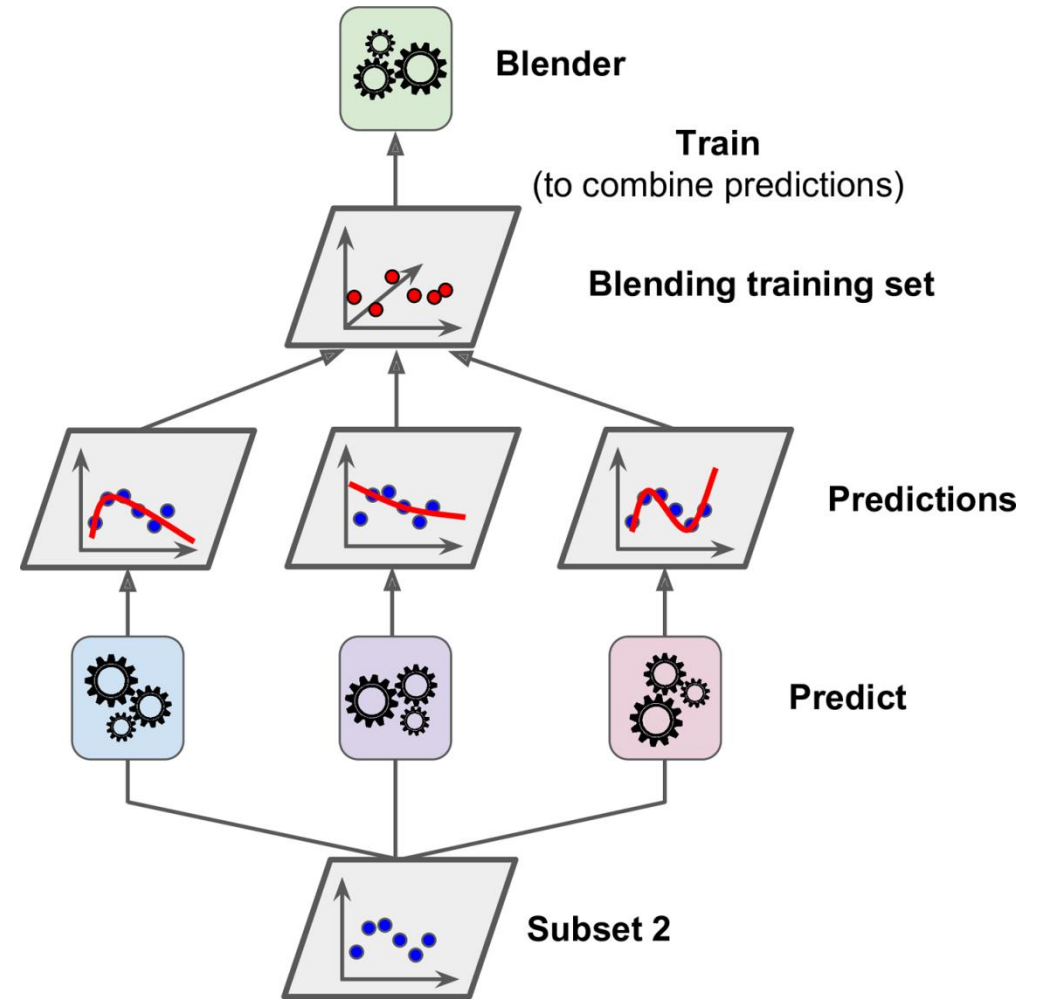
Stacking

- First, the training set is split into two subsets.
- The first subset is used to train the predictors in the first layer



Stacking

- Next, the first layer's predictors are used to make predictions on the second set
- For each instance in the hold-out set, there are three predicted values.
- We can create a new training set using these predicted values as input features and keeping the target values



Ensemble Learning – Pros and Cons

- Pros:
 - Higher accuracy- often the winners of Machine Learning challenges
 - With Bagging, you can limit error from variance
 - With Boosting, you can limit error from bias
 - With Stacking, you can limit variance and bias error while improving accuracy
- Cons:
 - Can be harder to interpret
 - More difficult to create and train
 - More difficult to DEPLOY in production (key reason why it's not widespread in industry)

